

1. Perceptron

(1) Decision Process

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \xrightarrow{\text{input}} z = \vec{w}^T x + b \xrightarrow{\text{Unit Step function}} \phi(z) = \begin{cases} 1 & z \geq \theta = 0 \\ -1 & z < \theta = 0 \end{cases}$$

② Initialize $\vec{w}, b$

(2) Parameters Updation.

① Set Hyperparameter:

Learning Rate η ③ $\hat{y}^{(i)}$: i -th prediction label; $y^{(i)}$: i -th true label

$$\begin{cases} \Delta \vec{w} = \eta (y^{(i)} - \hat{y}^{(i)}) \vec{x}^{(i)} \in \{\pm 2\eta x^{(i)}, 0\} \\ \vec{w} \rightarrow \vec{w} + \Delta \vec{w} \\ \Delta b = \eta (y^{(i)} - \hat{y}^{(i)}) \\ b \rightarrow b + \Delta b \end{cases}$$

(3) Convergence: ① Linear Separable.

② If not ①, never stop updation unless set epochs/max. iter.

2. Adaptive Linear Neuron (Adaline)

(1) Prediction process same as Perceptron.

(2) Cost Function: $\hat{y}^{(i)} = \phi(\vec{w}^T \vec{x}^{(i)} + b)$ ① Squared Error: $L^{(i)} = \frac{1}{2} (y^{(i)} - \hat{y}^{(i)})^2$ ② Sum of SE (SSE): $J = \sum L^{(i)} \rightarrow$ Used in Slides③ Mean SE (MSE): $\bar{L} = \frac{1}{n} \text{SSE} \rightarrow$ Used in practiceNote that MSE $\rightarrow$ convex and derivable(3) Parameter Updation: $\rightarrow$ MSE.

Move follow the gradient's negative direction

$$\nabla L^{(i)}(\vec{w}) = (y^{(i)} - \hat{y}^{(i)}) \vec{x}^{(i)}$$

① Stochastic Gradient Descent (SGD):

Update for each sample

② Minibatch GD (Mini BGD):

Update for each mini batch with m samples.

$$\Delta \vec{w} = \eta \cdot \nabla \text{MSE}(\vec{w}) = \eta \cdot \frac{1}{m} \sum (y^{(i)} - \hat{y}^{(i)}) \vec{x}^{(i)}$$

$$\Delta b = \eta \cdot \frac{1}{m} \sum (y^{(i)} - \hat{y}^{(i)})$$

Choice of b : 32 $\rightarrow$ 64 $\rightarrow$ 128 ...

③ Batch GD (BGD):

 $b = n$ Data need pre-standardized!

3. Logistic Regression

(1) Prediction:

$$z = \vec{w}^T \vec{x} + b \rightarrow \begin{cases} \phi(z) \rightarrow 1 & \text{when } z \rightarrow +\infty \\ \phi(z) \rightarrow 0 & \text{when } z \rightarrow -\infty \end{cases}$$

$$P(y=1 | \vec{x}, \vec{w}) = \phi(z) = \frac{1}{1 + e^{-z}} \rightarrow \text{sigmoid function}$$

$$\hat{y} = \begin{cases} 1 & \text{when } P > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

(2) Likelihood. We have n observation $s_1, s_2, \dots, s_n$.

$$L = \prod P(y=x_i | \theta), \quad \bar{L} = \log L = \sum \log P(y=x_i | \theta)$$

(3) Cost function: Max Likelihood Estimation (MLE)

For LR, we have:

$$L^{(i)} = L(y | \vec{x}^{(i)}, \vec{w}) = \phi(z) (1 - \phi(z))^{1 - y^{(i)}}$$

$$\bar{L}^{(i)} = \log L^{(i)} = y^{(i)} \log \phi(z) + (1 - y^{(i)}) \log (1 - \phi(z))$$

So the total cost of a batch of m samples is:

$$J(\vec{w}, b) = -\frac{1}{m} \sum \bar{L}^{(i)}$$

$$\nabla J(\vec{w}) = -\frac{1}{m} \sum (y^{(i)} - \hat{y}^{(i)}) \vec{x}^{(i)}$$

$$\frac{\partial J}{\partial \vec{w}} = -\frac{1}{m} \sum (y^{(i)} - \hat{y}^{(i)}) \vec{x}^{(i)}, \quad \Delta \vec{w} = \eta \nabla J(\vec{w})$$

$$\frac{\partial J}{\partial b} = -\frac{1}{m} \sum (y^{(i)} - \hat{y}^{(i)}), \quad \Delta b = \eta \frac{\partial J}{\partial b}$$

(5) multi-class LR

$$z_k = \vec{w}_k^T \vec{x} + b_k$$

$$P(y=k | \vec{w}, b) = \frac{e^{z_k}}{\sum e^{z_k}}$$

$$\bar{L} = -\sum_{k=1}^K y_k \log P(y=k | \vec{x})$$

(6) Pros: ① Simpler ② Online Updation ③ Probability

Cons: ① Sensitive to Outliers

4. K-Nearest Neighbor (KNN)

Prediction $\rightarrow$ the majority vote of KNN of x Min Kowski Distance: $\rightarrow$ bias $\rightarrow$ measure fitting

$$D(x, y) = (\sum |x_i - y_i|^p)^{1/p} \rightarrow \text{variance} \rightarrow \text{sensitive to noisy}$$

$$K \uparrow, \text{bias} \uparrow, \text{variance} \downarrow$$

(2) 3 types: ① ID3: ② ID4: ③ ID5:

① Iterative Dichotomiser 3 (ID3):

Find the feature divide to maximize Info Gain

② Classifier Version 4 Improvement (C4.5):

Gain Ratio

③ Classification And Regression Trees (CART)

Binary Tree, maximize Gini Gain

(3) Random forest: Bootstrap + DT

5. Support Vector Machine

(1) Prediction: $\hat{y}_i, z^{(i)} \geq 1, \hat{y}_i \in \{1, -1\}$ (2) Core Idea: Maximize width of "street".
margin: distance between hyperplane and the nearest data point.

$$\text{width} = (x_1 - x_2)^T \frac{\vec{w}}{\|\vec{w}\|} = \frac{2}{\|\vec{w}\|}$$

where $\vec{w}^T x_1 + b \geq 1, \vec{w}^T x_2 + b \leq -1$ $\therefore$ maximize $\frac{2}{\|\vec{w}\|} \leftrightarrow$ minimize $\|\vec{w}\|$ i.e. $\min_{\vec{w}, b} \frac{1}{2} \|\vec{w}\|^2, \text{ s.t. } \hat{y}^{(i)} (\vec{w}^T x^{(i)} + b) \geq 1$ Convex $\rightarrow$ never local minima(3) Kernel Trick: $\rightarrow$ other inner product.① Linear $k(x, y) = x^T y$ ② Polynomial: $k(x, y) = (x^T y + c)^d$

③ Radial basis function (RBF)/Gaussian:

$$k(x, y) = e^{-\gamma \|x - y\|^2}$$

④ Sigmoid: $k(x, y) = \tanh(\alpha x^T y + c)$

(4) Soft Margin:

$$\min_{\vec{w}, b, \xi} \frac{1}{2} \|\vec{w}\|^2 + C \sum \xi, \text{ s.t. } y_i (\vec{w}^T x_i + b) \geq 1 - \xi; \text{ for all } i$$

* 若 unfitted, ξ 大, C 小

6. Decision Tree. Pros: Interpretability

(1) Definitions.

① Information Entropy: $I_H(x) = -\sum p_i \log_2 p_i$ ② Info-Gain: $I_G(D, f) = I_H(D) - \sum \frac{|D_i|}{|D|} I_H(D_i)$ where D - dataset; f $\rightarrow$ classification method; D_i : dataset of i th subnode③ Gini Impurity: Describe the impurity of D .

$$I_G(D) = \sum p(i|D) (1 - p(i|D)) = 1 - \sum p(i|D)^2$$

④ Gini Gain: $\text{GiniGain}(D, f) = I_G(D) - \sum \frac{|D_i|}{|D|} I_G(D_i)$ ⑤ Classification Error: $I_E(D) = 1 - \max p(i|D)$ ⑥ Intrinsic Value: $I_V(f) = -\sum \frac{|D_i|}{|D|} \log_2 \frac{|D_i|}{|D|}$ ⑦ Gain Ratio: $\text{GainRatio}(D, f) = \frac{I_G(D, f)}{I_V(f)}$

7. Data Preprocessing

(1) Missing Data: ① Remove the whole row/column ② Interpolation with

a. Mean b. Median c. Most frequent d. Consensus e. KNN

(2) Categorical Data:

① Label Encoding: low $\rightarrow$ 0, medium $\rightarrow$ 1, high $\rightarrow$ 2 ...② One-hot Encoding: red $\rightarrow$ [1, 0, 0] blue [0, 1, 0] ...(3) Partitioning: Data $\rightarrow$ training - validation - test.

(4) Feature Scaling:

① Standardization (Z-score Normalization)

$$x_{std} = \frac{x - \mu}{\sigma}, \mu \rightarrow \text{mean}, \sigma \rightarrow \text{standard deviation}$$

② Min-Max Scaling (Norm) $x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$

③ Robust Scaling:

$$x_{robust} = \frac{x - \text{median}}{IQR}, IQR: \text{Interquartile range } IQR = Q_3 - Q_1$$

(5) Feature Selection: ① L1-L2: $w_{\text{feature}} \rightarrow 0$ L1: constant line; $\sum |w_i| \leq c \rightarrow$ diamond; L2: $\sum w_i^2 \leq c \rightarrow$ circle.

② Sequential Backward Selection (SBS): Try del. evaluate. greedily

③ Random forest.

8. Dimensionality Reduction

(1) Projection: ① Principal Component Analysis (PCA)

Mathematically, PCA rotate coordinate system, s.t. first k

PCs contain the most info (with most variance), then project.

a. Standardize $\vec{X}$ b. Compute Covariance matrix

c. Compute eigen value/vectors d. sort by eigenvalue

e. Project data onto new subspace.

(2) Manifold Learning: Assume data lies on low-dim manifold

① Locally Linear Embedding $\rightarrow$ preserve the intrinsic geometric

• Step 1: Find Neighbors

• For each data point, identify its k nearest neighbors (using Euclidean distance).

• This defines the local geometry around each point.

• Step 2: Reconstruct with Weights

• Calculate a set of optimal weights (W) that best reconstructs each point as a linear combination of its neighbors.

• The goal is to minimize the reconstruction error using the same weights.

• These weights capture the local geometric properties.

• Step 3: Low-Dimensional Embedding

• Map the data to a low-dimensional space (e.g., 2D or 3D).

• Crucial Step: Keep the weights (W) fixed (from Step 2) and find new coordinates (y) that minimize the reconstruction error using the same weights.

• This ensures that the local relationships learned in high dimensions are preserved in the low-dimensional map.

② T-Distributed Stochastic Neighbor Embedding

Idea: A on B classes on high-dim $\rightarrow$ same on low-dim.

a. Compute Pairwise Similarities.

b. Define "similar measure" in low-dim

c. Minimize the Kullback-Leibler divergence.

8. Evaluation and Tuning

- (1) Validation ① Hand-out: Partition data into {training, validation}
- ② k-fold cross: Data $\rightarrow$ k subset (fold), repeat k-times:
- use $k-1 \rightarrow$ train, $1 \rightarrow$ validation. Return average ~
- $n \downarrow, k \uparrow \rightarrow X_i$

③ Leave One Out Cross Validation (LOOCV) $\rightarrow k = N_{\text{sample}}$

④ Simplified k-fold cross: Ensure distribution same for all fold

(2) Learning Curves: function, $y \rightarrow$ Accuracy, $x \rightarrow$ num of samples

(3) Hyperparameter Tuning

- ① Grid Search: Brutely enumerate
- ② Random Search: Randomly select, more efficient
- ③ Bayesian Optimization:

Nested-cross Validation: On each training set (with $k-1$ folds) do k-fold to tuning and model select

(4) Evaluation

True Positoin (TP) $\rightarrow$ True Label, correct predicted.

① Error: $ERR = \frac{F}{\text{Total}}$ ② Accuracy: $ACC = \frac{T}{\text{Total}}$

③ TP-Ratio: $TPR = \frac{TP}{TP+FN}$ ④ TNR: $TNR = \frac{TN}{TN+FP}$

⑤ FPR: $FPR = \frac{FP}{TN+FP}$ ⑥ FNR: $FNR = \frac{FN}{TP+FN}$

⑦ Precision: $PRE = \frac{TP}{P}$ ⑧ Recall: $REC = \frac{TP}{T}$

⑨ F1-Score: $F_1 = 2 \cdot \frac{PRE \cdot REC}{PRE + REC}$

⑩ Receiver Operating Characteristic (ROC) Curve: $y \rightarrow$ Recall, $x \rightarrow$ FPR

⑪ Area Under the Curve (AUC): Area of ROC $\rightarrow AUC = \int_0^1 REC(FPR) dFPR$

For Multiclass classification:

① Macro-average: e.g. $TP_{\text{macro}} = \frac{\sum TP_i}{n}$, so does TN, FPR, FNR

② Micro-average: e.g. $TP_{\text{micro}} = \frac{\sum TP_i}{\sum P_i}$ ③ Weighted

(5) Imbalanced Classification

① Upsampling: Sampling for minority class

② Down sampling: ~

10. Ensemble Learning

(1) Majority $\rightarrow$ Weighted voting

(2) Bagging (Bootstrap Aggregating)

① Bootstrap Sample $\rightarrow$ Create multiple subset. some appear multiple

② Parallel Training: on each subset ③ Majority

(3) Boosting: ① Init sample-weights $w_i = \frac{1}{n}$ ② m iteration, for j-th:

a. train $C_j(x, y, w)$ b. w-error rate: $\epsilon_j = \sum w_i$

c. Classifier coeff: $\alpha_j = \frac{1}{2} \ln \frac{1-\epsilon_j}{\epsilon_j}$ d. $w \rightarrow w \times e^{-\alpha_j y_j}$ e. $w \rightarrow w \times \epsilon_j$

After all, predict by voting.

9. Clustering

(1) Two types: ① Partition $\sim$: Divide into non-overlapping subsets.

② Hierarchical $\sim$: Build a tree.

Well-separated Clusters (分离良好的簇类):

(2) Center-based Clustering (基于中心的聚类):

• Clusters are defined by a central point (e.g., K-means).

• The data points in a cluster are closer to the cluster center than to any other cluster center.

Continuous Clustering (连续模型聚类):

• Clusters are defined by connected regions in the feature space (e.g., DBSCAN).

• The data points in a cluster are connected to each other by paths of neighboring points.

Density-based Clustering (基于密度的聚类):

• Clusters are defined as areas of higher density than the remainder of the data set (e.g., DBSCAN).

• Clusters are formed by connecting areas of high density, and noise points are those that do not belong to any cluster.

Objective Function Clustering (基于目标函数的聚类):

• Clusters are defined by optimizing a specific objective function (e.g., K-means minimizes the within-cluster sum of squares).

• The goal is to find a partition of the data that minimizes (or maximizes) the objective function.

• Local Objective Function (局部目标函数):

• Depends only on the data points in the same cluster (e.g., K-means).

• Global Objective Function (全局目标函数):

• Depends on the data points in all clusters (e.g., Spectral Clustering).

(3) Algorithms:

① K-means $\rightarrow$ Partition, center-based.

a. Select k points as initial centroid

Random select; multiple run for it; k-means++ $\rightarrow$ $\text{RMSE} > K$, fix it

b. Post-processing: After all, rerun for bad clusters.

Bisecting k-means. $\approx 2 \times k=1$, repeatedly split highest SSE into 2.

Repeat:

b. For each x, assign to nearest clauster

c. Update centroid to mean of data.

① Standard: update each iteration ① Online: ~ each sample.

O(nkl). n $\rightarrow$ samples, k $\rightarrow$ clusters, l $\rightarrow$ iteration, d $\rightarrow$ dimension

② Hierarchical ~

a. Agglomerative HC $\rightarrow$ From the bottom up

(a) Regard each point as a cluster

(b) Repeat: Compute distance between all pair of clusters by:

③ Single Linkage $\rightarrow$ minimum distance

④ Complete Linkage $\rightarrow$ max ~

⑤ Average distance ⑥ Centroid dist

⑦ Ward's Method $\rightarrow$ Increase of SSE if merge

and then merge them.

b. Divisive HC $\rightarrow$ From the top down.

(a) Regard all point in a cluster

(b) Repeat: Choose a cluster by:

⑧ Highest SSE ⑨ Biggest size ⑩ Larger diameter

and then split it by K-means or others.

c. MST HC

(a) Build a MST by distance

(b) Remove k-1 longest edges.

11. Regression

11.1 Linear Regression

Simple LR $\rightarrow y = wx + b$, minimize MSE of $\hat{y}$.

Multiple LR $\rightarrow y = \tilde{w}^T \tilde{x} + b$, minimize MSE of $\hat{y}$.

Exploratory Data Analysis:

Covariance: $Cov(X, Y) = \frac{1}{n} \sum (x_i - \bar{x})(y_i - \bar{y})$

Covariance coefficient: $\frac{Cov(X, Y)}{\sigma_X \sigma_Y} \rightarrow$ standard deviation.

Algorithm: ① Ordinary Least Square $\rightarrow$ minimize SSE

Similar to Adaline, we gradient descent, sensitive to outlier

② Robust Regression $\rightarrow$ implemented by Random Sample

Ignores outliers by fitting model only by inliers.

③ Randomly select minimum points ④ Fit them.

⑤ Select the point that distance in threshold, refit them

11.2. Model Evaluation:

① Mean Absolute Error: $MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$

② M-Squared Error: $MSE = \frac{1}{n} \sum |y_i - \hat{y}_i|^2$

③ R² Score: $R^2 = 1 - \frac{SSE}{SST} \rightarrow$ Total SS = $\sum (y_i - \bar{y})^2$

11.3. Regularization

Ridge (L₂): $J(w, b) = MSE + \lambda ||w||^2$

Lasso (L₁): $J(w, b) = MSE + \lambda \sum w_i$

Elastic Net: combine L₁ and L₂

11.4 Polynomial Regression

$y = W^T \tilde{x} + b$, may overfit when d $\uparrow$

11.5 Tree-based Regression

(1) Decision Tree R $\rightarrow$ split data, minimize MSE each node

(2) Random Forest R $\rightarrow$ ensemble DTR.

13. CNN

1. Limitations of MLP & CNN Core Concepts (MLP 局限与 CNN 引入)

① MLP 面对图像的三大死穴:

1. 参数数量爆炸 (Parameter Explosion): 若处理 $1000 \times 1000 \times 3$ 的大图, 仅隐藏层包含 1000 个神经元, 全连接参数就会瞬间突破 302, 导致彻底内存崩溃和灾难性过拟合。

2. 空间结构丢失 (Loss of Spatial Topology): MLP 强将 2D 矩阵图像展平 (Flatten) 为 1D 向量, 彻底割裂了像素点在上下左右邻域的空间位置关联。

② CNN 的三大制胜法宝 (Core Philosophies):

① 局部连接 (Local Connectivity): 每个神经元只关注图像的局部小区域 (感受野 Receptive Field), 不再全盘连接。

② 权重共享 (Parameter Sharing): 同一个滤波器 (卷积核 Filter) 在整个图像的所有空间位置上滑动复用同一组权重参数。

③ 平移不变性 (Translation Invariance): 无论特征出现在图像的左上角还是右下角, 经过相同的滤波器后都能提取出相似的特征响应。

2. Convolutional Layer Mechanics (卷积层运算与计算大题 100% 必考)

① 运作机制: 深度为 C_{in} 的三维输入特征图, 被一个大小为 $F \times F \times C_{in}$ 的滤波器进行滑动计算。在每个位置执行对应元素相乘并求和 (Dot Product), 最终加上一个共享偏置, 输出一张二维的激活图 (Activation Map)。如果有 C_{out} 个不同的滤波器, 就能输出深度为 C_{out} 的新三维特征图。

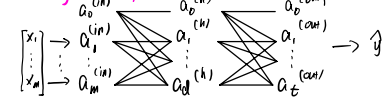
② 核心公式 1: 输出特征图尺寸计算 (Output Size Formula):

假设输入宽度/高度为 W , 滤波器大小为 F , 边界填充 Padding 数量为 P , 滑动步长 Stride 为 S , 则输出的单边尺寸 O 为:

$$O = \frac{W - F + 2P}{S} + 1$$

③ 考试重要提醒: 此公式计算结果必须为整数。若算出来是小数, 说明步长 S 无法整除, 在实际框架中会引发舍弃 (向下取整) 或报错。

12. Multilayer Artificial Neural Network



① $\tilde{x}^{(1)} = W_{ij}^{(1)}$ ② $\tilde{y}^{(2)} = (b_i^{(2)}, \phi(\tilde{x}^{(1)}))$

1. 从单层感知机到多层感知机 (MLP)

① 单层回顾 (Single-layer Recap): 如 Perceptron 和 Adaline, 只能解决线性可分 (Linearly Separable) 问题。面对简单的 XOR (异或门) 问题时彻底失效。 PDF

② 多层感知机架构 (Multilayer Perceptron - MLP): PDF

③ 至少包含一个隐藏层 (Hidden Layer)。 PDF

④ 属于典型的全连接前向神经网络 (Fully Connected Feedforward NN)。 PDF

⑤ 非凸代价函数 (Non-convex Cost Function): 多层非线性网络的代价函数布满了局部极小值 (Local Minima) 和鞍点。此时, 随机梯度下降 (SGD) 引入的随机噪声 (Noisy Nature) 反而因祸得福, 能帮助模型极好地跳出局部泥潭。 PDF

2. Forward Propagation & Activation Functions (前向传播与激活函数)

① Forward Propagation (前向传播流程):

输入向量 $x \rightarrow$ 与隐藏层权重矩阵相乘并加偏置 $\rightarrow$ 经过隐藏层激活函数 $\rightarrow$ 与输出层权重相乘并加偏置 $\rightarrow$ 经输出层激活函数得到最终预测 $\hat{y}$ 。 PDF

② 必考题: 为什么激活函数必须是而非线性的?

③ 如果隐藏层使用线性激活函数 (即 $\phi(z) = z$), 根据矩阵乘法的结合律, 多层线性变换复合起来的结果等价于单层的线性变换。网络会退化回单层感知机, 彻底失去深度学习拟合复杂非线性决策边界的能力。

④ 三大主流激活函数对比大礼包:

⑤ Sigmoid: $\phi(z) = \frac{1}{1+e^{-z}}$, 输出区间 $(0, 1)$, 缺点: 两端饱和、梯度极小, 极易导致梯度消失 (Vanishing Gradient); 非零中心化 (Not zero-centered)。

⑥ Tanh (双曲正切): $\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, 输出区间 $(-1, 1)$, 优势: 实现了零中心化 (Zero-centered), 收敛速度通常快于 Sigmoid, 但仍存在两端梯度消失问题。

⑦ ReLU (修正线性单元): $\phi(z) = \max(0, z)$, 绝对统治级优势: 在 $z > 0$ 区域梯度恒为 1, 彻底解决了正向的梯度消失问题, 且计算极其简单高效。缺点: 存在 Dying ReLU (神经元坏死) 现象, 一旦输入 $z < 0$ 梯度直接归零, 该神经元可能永久失效。

3. Backpropagation Algorithm (反向传播算法核心推导 100% 必考)

① 核心数学工具: 多元微积分的链式法则 (Chain Rule)。

② 算法目的: 将输出层计算出的损失误差 (Loss/Error), 从后往前层层倒推回各个隐藏层, 高效计算出代价函数对全网每一个权重 w_{ij} 和偏置 b_i 的偏导数。

③ 极简四步推导链条 (以一个简单节点为例, 设输入为 x , 隐藏层净输入为 z_h , 激活为 a_h , 输出层净输入为 z_o , 预测输出为 $\hat{y}$, 损失为 L):

1. 计算输出层误差项: $\delta_o = \frac{\partial L}{\partial \hat{y}} \cdot \phi'(z_o)$

2. 将误差项反向传回隐藏层 (乘以权重转置并乘隐藏层激活函数导数): $\delta_h = (\delta_o \cdot w_{ho}) \times \phi'(z_h)$

3. 计算相对于权重的总梯度: $\frac{\partial L}{\partial w_{ho}} = \delta_h \cdot \delta_o$

4. 根据梯度更新权重: $w := w - \eta \frac{\partial L}{\partial w}$ PDF

3. Pooling & Fully Connected Layers (池化层与全连接层)

① Pooling Layer (池化层/下采样):

② 两大主流: 最大化池 (Max Pooling) 以及平均池化 (Average Pooling)。

③ 绝对特性 (判断超参数考点): 池化层完全没有任何需要学习的权重参数 (Parameters = 0), 它只负责按照固定规则进行数据压缩, 用来前减特征图尺寸、减少算力消耗并控制过拟合。

④ Fully Connected Layer (全连接层/FC):

⑤ 通常置于 CNN 的末端, 将前面提取出的多维特征图彻底展平为一维长向量, 整合所有高阶特征, 输入至 Softmax 层输出最终的类别分类得分。

4. Classic CNN Case Studies (经典网络演进与考点速查)

经典模型 (Model) | 核心历史贡献与期末高频考点 (Key Innovations & Exam Focus)

AlexNet (2012) | 现代深度学习的开山之作, 首次成功应用 ReLU 激活函数 (加速收敛); 引入 Dropout 防止过拟合; 全面开启 GPU 硬件加速时代。

VGG (2014) | 奠定了“越深越好”的基础, 提出只使用极简单核的 3×3 小卷积核进行多堆叠。

⑥ 经典考点: 两个 3×3 卷积核连续堆叠, 其有效感受野 (Receptive Field) 等价于一个 5×5 的卷积核, 但前者参数更少且引入了更多非线性激活!

GoogLeNet (2014) | 极致压榨计算效率, 首创 Inception 模块, 并行进行多尺度特征提取: 大量引入 1×1 瓶颈卷积 (Bottleneck Convolution) 专门进行降维与通道压缩; 用全局平均池化 (Global Average Pooling) 彻底取代了极为笨重、肿胀的 FC 全连接层, 使总参数量暴跌。

ResNet (2015) | 革命性解决退化问题, 引入了残差连接/跳跃连接 (Skip Connections / Shortcut), 强行构建恒等映射方程 $H(x) = F(x) + x$, 彻底斩断了反向传播中的梯度消失链条, 使得训练几百层、上千层的超级级神经网络成为现实。